

Module 08 Combining and Sorting Data

IBM InfoSphere Information Server for Data Integration

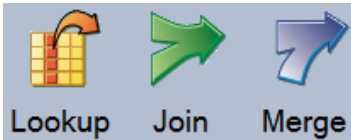
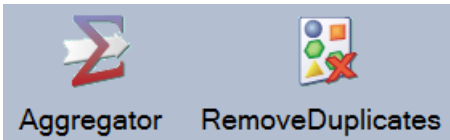


Module Objectives

After completing this topic, you should be able to:

- Combine data using the Lookup stage
- Define range lookups
- Sort Data using Sort Stage
- Remove duplicates using Remove Duplicate Stage
- Combine data using the Join stage
- Combine data using the Merge stage
- Combine data using the Funnel stage

Ways of Combining Data

Property	Horizontally	Vertically	Funneling
Number of input streams	Multiple Input Links	One input link	Multiple input streams
Number of output streams	One output link made of columns from different input links	one output link combining groups of related records into a single record	single output stream
Example	Examples:  Lookup Join Merge	Examples:  Aggregator RemoveDuplicates	Example :  Funnel

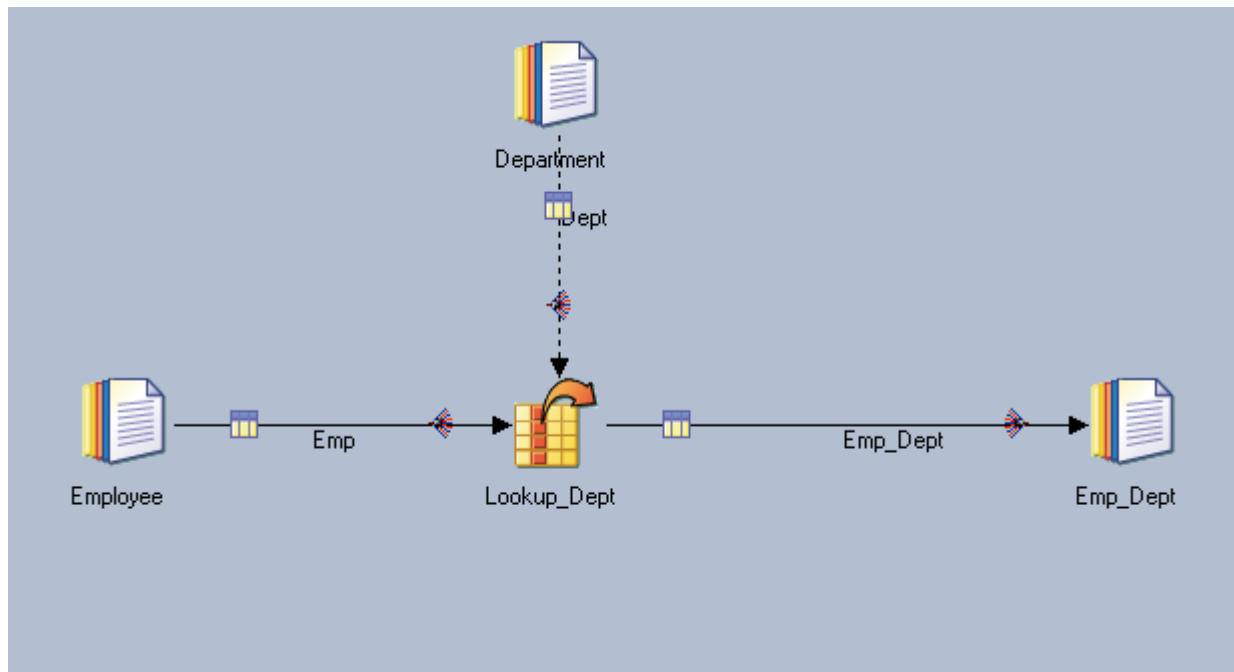
Lookup Stage

- LOOKUP matches records in its primary input stream with records in its referenced input streams specified by the reference links.
- The records are matched on the basis of a key field
- It combines matched records with selected columns from the multiple input streams and write to a single output stream.
- Unmatched input records are handled as specified in the option. The records are matched on the basis of a key field.

Lookup Features

- The most common use for a lookup is to map short codes from input data onto expanded information from a lookup table which is then joined to the incoming data, then output
- One stream input link (source link)
- Multiple reference links
- One output link
- Lookup failure options
 - Continue, Drop, Fail, Reject
- Reject link
 - Only available with Reject lookup failure option
- Can return multiple matching rows
- Look up table is loaded into memory before the job get started
- Hash table is built in memory from the lookup tables
 - Indexed by key
 - Needs to fit into physical memory
- By default, the partitioning method for Lookup on reference links is Entire

Lookup Job Example



Example of Lookup

Input Data

Emp name	Emp number	Base location	Zip Code (Key)
Sulekha	123476	Pune	411006
Vinod	447799	Pune	411057
Albert	7765	Bangalore	560002
Sudhir	975433	Pune	411028

Lookup Data

Zip Code (Key)	Addr1	Addr2
411006	IBM PUNE, Next to DON BOSCO High School	off airport road Yerwada
411028	IBM Pune,Quadra Office	Near Magarpatta City, Pune
411057	IBM Pune, ETZ Office	Marunji Pune, Maharashtra

Lookup Failure Actions

- If the lookup fails to find a matching key from the lookup table for a specific source record, one of several actions can be taken
- Fail (Default)
 - Stage reports an error and the job fails immediately
- Drop → like inner join
 - Input row is dropped
- Continue → like left join
 - Input row is transferred to the output. Reference link columns are filled with null or default values
- Reject
 - Input row sent to a reject link
 - This requires that a reject link has been created for the stage

Specifying Lookup Failure option in the Job

The screenshot shows the 'Lookup_3 - Lookup Stage' configuration window. A yellow callout points to the 'Lookup constraints' section, which includes a table for defining key expressions. Another yellow callout points to the 'Multiple rows returned from link:' dropdown, which is set to 'Dept'. A third yellow callout points to the 'Lookup Failure' dropdown menu, which is open and shows options: 'Fail', 'Continue', 'Drop', 'Fail', and 'Reject'. A fourth yellow callout points to the 'Fail' option in the dropdown menu.

Lookup constraints

Key Expression	Range	Column Name
	☐	EmpID

Select to return multiple rows

Multiple rows returned from link:
Dept

	Link Name	Condition	Condition Not Met	Lookup Failure
1	Dept	Emp.DeptID	Fail	Fail

Select the failure options

Fail
Continue
Drop
Fail
Reject

OK Cancel Help

Lookup Stage

- Output of Lookup with Continue option → like left join

Employee Name	Employee Number	Base location	Address1	Address2	Zip Code
Sulekha	123476	Pune	IBM PUNE, Next to DON BOSCO High School ,	off airport road Yerwada	411006
Vinod	447799	Pune	IBM Pune, ETZ Office	Marunji Pune, Maharashtra	411057
Albert	7763	Bangalore			560002
Sudhir	975433	Pune	IBM Pune, Quadra Office	Near Magarpatta City, Pune	411028

Lookup Stage

- Output of Lookup with Drop option → like inner join

Employee Name	Employee Number	Base location	Address1	Address2	Zip Code
Sulekha	123476	Pune	IBM PUNE, Next to DON BOSCO High School ,	off airport road Yerwada	411006
Vinod	447799	Pune	IBM Pune, ETZ Office	Marunji Pune, Maharashtra	411057
Sudhir	975433	Pune	IBM Pune, Quadra Office	Near Magarpatta City, Pune	411028

Lookup Conditions

- Equality match
 - Match exact values in the lookup key column of the reference link to selected values in the source row
 - Return row or rows (if multiple matches are to be returned) that match
- Caseless match
 - Like an equality match; ignores case e.g., “abc” matches “AbC”
- Range lookup
 - Define the range by selecting the upper and lower bound range columns (e.g.
`Account_Detail.Trans_Date >= Customer_Detail.Start_Date AND Account_Detail.Trans_Date <= Customer_Detail.End_Date`)
 - Range on the reference link
 - Two columns on the reference link define the range
 - A match occurs when a selected value in the source row is within the range
 - Range on the source link
 - Two columns on the source link define the range
 - A match occurs when a selected value in the reference link is within the range

Lookup Stage with Equality Match

Column Auto-Match

Mappings to output columns

Reference key column and source mapping

Column definitions

The screenshot shows the 'Lookup_Dept - Lookup Stage' configuration window. It features three main tables: 'Emp', 'Dept', and 'Emp_Dept'. The 'Emp' table has columns 'Key Expression', 'Range', and 'Column Name'. The 'Dept' table has columns 'Key Expression', 'Key Type', and 'Column Name'. The 'Emp_Dept' table has columns 'Derivation' and 'Column Name'. Below these tables are two sections for column definitions: 'Emp' and 'Dept' on the left, and 'Emp_Dept' on the right. The 'Emp' section shows 'DeptID' as a key column. The 'Dept' section shows 'Emp.DeptID' as a key expression. The 'Emp_Dept' section shows 'EmpID' as a key column. Annotations highlight the 'Column Auto-Match' feature, the 'Mappings to output columns' section, the 'Reference key column and source mapping' section, and the 'Column definitions' section.

Emp		
Key Expression	Range	Column Name
	☒	EmpID
	☐	EmpName
	☐	Location
	☐	DeptID

Dept		
Key Expression	Key Type	Column Name
Emp.DeptID	=	DeptID
		DeptName

Emp_Dept	
Derivation	Column Name
Emp.EmpID	EmpID
Emp.EmpName	EmpName
Emp.Location	Location
Dept.DeptID	DeptID
Dept.DeptName	DeptName

Emp	Dept
Column name	Key
1 DeptID	☒
2 DeptName	☐

Emp_Dept		
Column name	Key	SQL type
1 EmpID	☒	Integer
2 EmpName	☐	VarChar
3 Location	☐	VarChar
4 DeptID	☐	Integer

Lookup Stage with Caseless Match

Lnk_Emp

Key Expression	Range	Column Name
	☒	EmpID
	☐	EmpName
	☐	Location
	☐	DeptID
	☐	DOB
	☐	DOJ

Lnk_Insurance

Condition	Key Expression	Key Type	Column Name
	Lnk_Emp.EmpID	Caseless	EmpID
			PolicyDate

Member

Derivation	Column Name
Lnk_Emp.EmpID	EmpID
Lnk_Emp.EmpName	EmpName
Lnk_Emp.DeptID	DeptID
Lnk_Emp.DOB	DOB
Lnk_Emp.DOJ	DOJ

Key Types

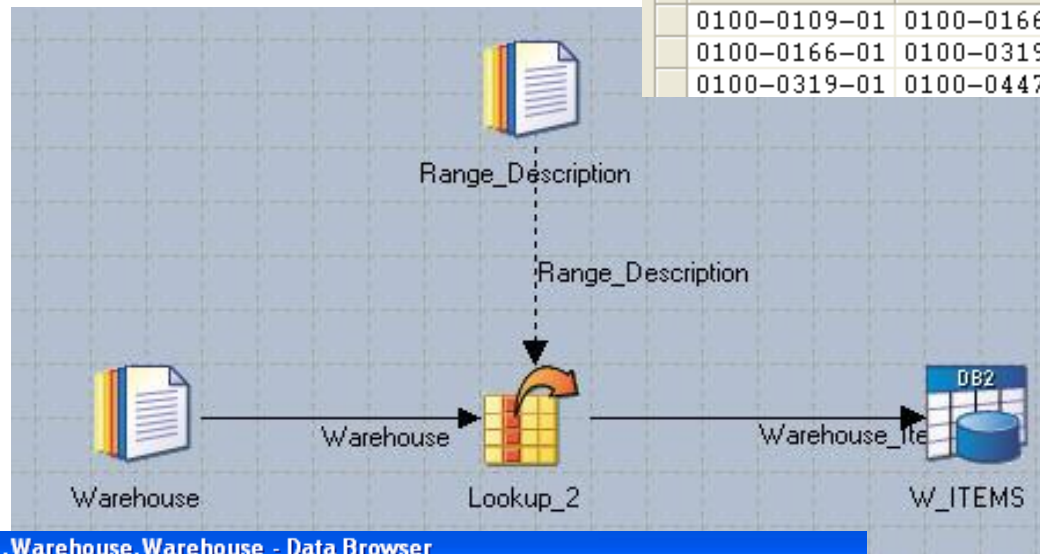
Range Lookup Job

- Range on Source Stream Link
 - Compares the value of a lookup column to a range of values between two stream columns
 - Supports multiple Ranges
- Range on Reference Link
 - Compares the values of a stream column to a range of values between two lookup columns

Range on Reference Link

Reference
range values

LookupItemDescription..Range_Description.Range_D			
StartItem	EndItem	Description	
0100-0109-01	0100-0166-01	Description A	
0100-0166-01	0100-0319-01	Description B	
0100-0319-01	0100-0447-01	Description C	



Retrieve
description

LookupItemDescription..Warehouse.Warehouse - Data Browser						
Warehouse	Item	Onhand	Onorder	Allocated	HardAllocated	
100	0100-0109-01	0474.000000	0030.000000	0131.000000	35.000000	
100	0100-0166-01	0094.000000	0059.000000	0047.000000	40.000000	
100	0100-0319-01	0003.000000	0000.000000	0000.000000	0.000000	

Source values

Specifying the Range Lookup

Double-click to specify range

Source link

Reference link

Warehouse

Key Expression	Range	Column Name
	☐	Warehouse
Range[Range_Description.StartItem,	☒	Item
	☐	Onhand
	☐	Onorder
	☐	Allocated
	☐	HardAllocated

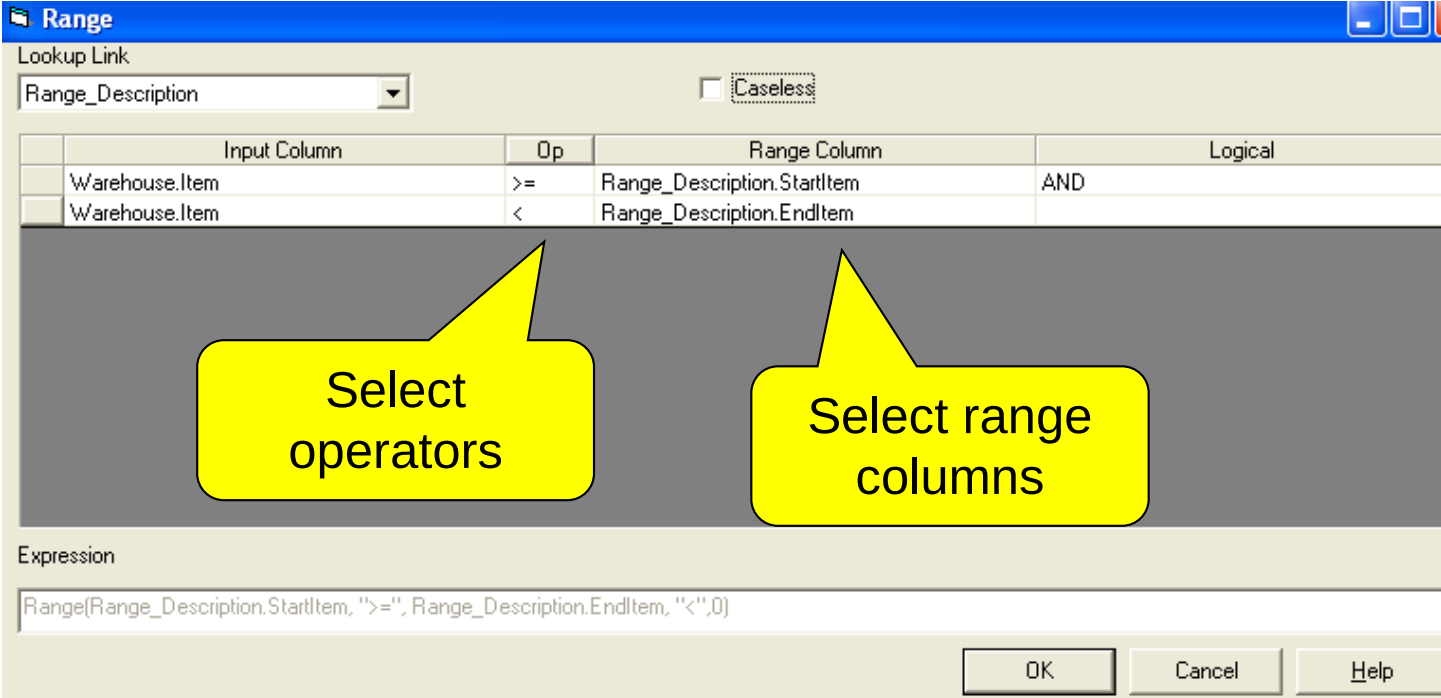
Range_Description

Key Expression	Key Type	Column Name
		StartItem
		EndItem
		Description

Warehouse_Items

Derivation	Column Name
Warehouse.Warehouse	Warehouse
Warehouse.Item	Item
Warehouse.Onhand	Onhand
Warehouse.Onorder	Onorder
Warehouse.Allocated	Allocated
Warehouse.HardAllocated	HardAllocated
Range_Description.Description	Description

Range Expression Editor



The Range Expression Editor dialog box is shown. It has a title bar 'Range' and standard window controls. Below the title bar is a 'Lookup Link' section with a dropdown menu set to 'Range_Description' and an unchecked 'Caseless' checkbox. A table with four columns (Input Column, Op, Range Column, Logical) contains two rows. The first row shows 'Warehouse.Item' with operator '>=' and range column 'Range_Description.StartItem'. The second row shows 'Warehouse.Item' with operator '<' and range column 'Range_Description.EndItem'. Below the table is a large grey area with two yellow callout boxes: 'Select operators' pointing to the 'Op' column and 'Select range columns' pointing to the 'Range Column' column. At the bottom is an 'Expression' text box containing the formula 'Range(Range_Description.StartItem, ">=", Range_Description.EndItem, "<", 0)'. The bottom right has 'OK', 'Cancel', and 'Help' buttons.

Range

Lookup Link

Range_Description ☐ Caseless

Input Column	Op	Range Column	Logical
Warehouse.Item	>=	Range_Description.StartItem	AND
Warehouse.Item	<	Range_Description.EndItem	

Select operators

Select range columns

Expression

Range(Range_Description.StartItem, ">=", Range_Description.EndItem, "<", 0)

OK Cancel Help

Range on Source Stream Link

Reference link key

Warehouse	Item	Onhand	Onorder	Allocated	HardAllocated
100	0100-0109-01	0474.000000	0030.000000	0131.000000	35.000000
100	0100-0166-01	0094.000000	0059.000000	0047.000000	40.000000
100	0100-0319-01	0003.000000	0000.000000	0000.000000	0.000000

Retrieve other column values

StartItem	EndItem	Description
0100-0109-01	0100-0166-01	Description A
0100-0166-01	0100-0319-01	Description B
0100-0319-01	0100-0447-01	Description C

Range values

Specifying the Range Lookup

Lookup_Item - Lookup Stage

Range_Description

Key Expression	Range	Column Name
	☐	StartItem
	☐	EndItem
	☐	Description

Warehouse

Condition:

Key Expression	Key Type	Column Name
Range[Range_Description.StartItem, ">=", Range_Description.EndItem, "<=", 0]	a..z	Item

Equality...
Caseless
Range

Reference link
key column
Range type

Warehouse_Items

Column name	Key	SQL type	Length	Scale	Nullable
1 Warehouse	☐	Integer	10		No
2 Item	☒	VarChar	50		No
3 Onhand	☐	VarChar	15		No
4 Onorder	☐	VarChar	15		No

Warehouse_Items

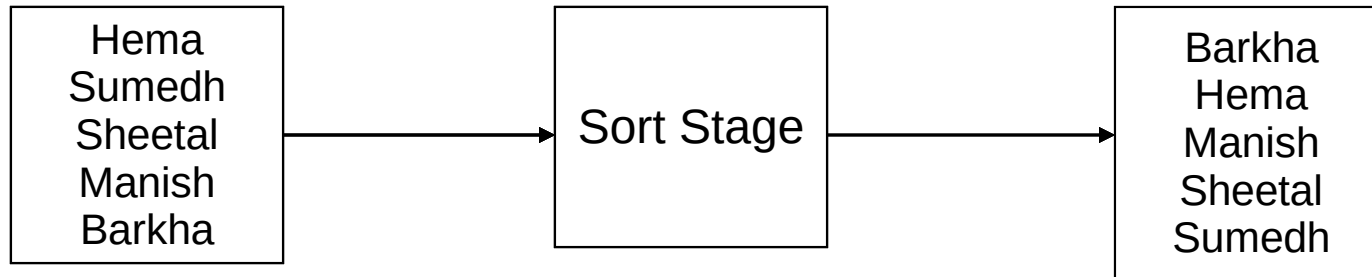
Column name	Key	SQL type	Length	Scale	Nullable
1 Warehouse	☐	Integer	10		No
2 Item	☒	VarChar	50		No
3 Onhand	☐	VarChar	15		No
4 Onorder	☐	VarChar	15		No

Properties of Sort

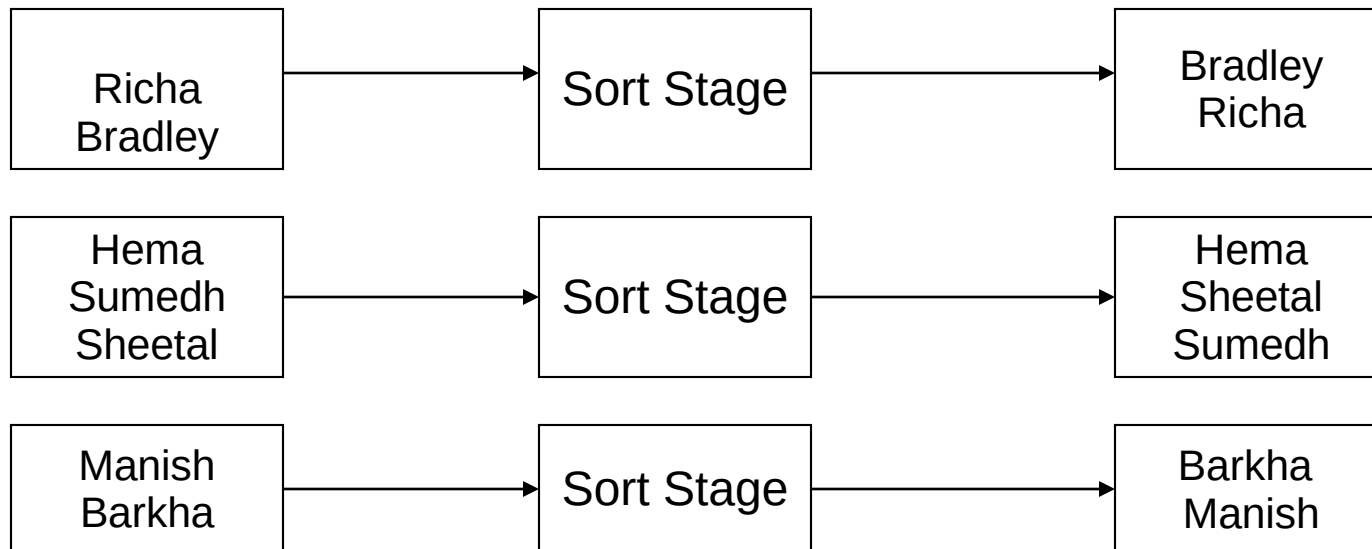
- Sorting can be done in **sequential mode** to sort entire data set **or in parallel mode** to sort within partitions
- Sort Stage uses temporary disk space (**scratch disk**) when performing Sort
- Use Same partitioning in case there is a possibility of the data sort order getting destroyed due to repartitioning
- Specify the primary sort keys and secondary sort keys
- Specify the sort order as ascending or descending
- Specify the sort as case sensitive or not

Sort Example

■ Sequential Mode



■ Parallel Mode



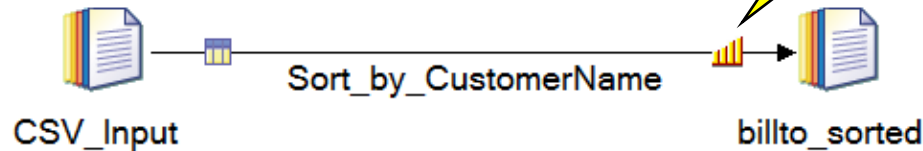
Sorting in DataStage

Using Sort Stage



Sort stage

In-Stage Sort



Sort within stage

The main difference between both is that with the [sort stage](#) you will be able to find much more sorting options than in the In-stage sort. You can have more control on your sort operation.

Sort Stage



- It has a single input link and a single output link carrying sorted data.
- Sorting is based on the defined Sort Key Column(s).

In-Stage Sorting

The screenshot shows the 'Merge_7 - Merge' dialog box with the 'Partitioning' tab selected. The 'Input name' is 'Warehouse'. The 'Partitioning / Collecting' section has 'Hash' selected for 'Partition type'. The 'Sorting' section has 'Perform sort' checked, with 'Stable' and 'Unique' options unchecked. The 'Available' list on the left includes 'Allocated', 'Onhand', 'Warehouse', 'HardAllocated', and 'Onorder'. The 'Selected' list on the right shows 'Item' as the sort key. Annotations highlight key settings for in-stage sorting.

Partitioning tab

Do sort

Preserve non-key col. ordering

Remove key dups

Can't be Auto when sorting in stage

Sort key

Key	Usage	Options
Item	Sorting, Partitioning	Ascending, Case sensitive,

Dedicated Sort Stage

In this example, the first 2 keys have already been sorted and our requirement asks us to maintain that sorting key order

Sort keys

Stage | Input | Output |

Stage name: Sort_Items

General | Properties | Advanced |

Sorting Keys

- Key = Item
 - Case Sensitive = False
 - Sort Key Mode = Don't Sort (Previously Sorted)
 - Sort Order = Ascending
- Key = Warehouse
 - Sort Key Mode = Don't Sort (Previously Sorted)
 - Sort Order = Ascending
- Key = fileNameColumn
 - Case Sensitive = False
 - Sort Key Mode = Sort
 - Sort Order = Descending

Options

- Allow Duplicates = True
- Create Cluster Key Change Column = False
- Create Key Change Column = False
- Output Statistics = False
- Restrict Memory Usage (MB) = 20
- Sort Utility = DataStage
- Stable Sort = True

Sort options

[No property selected]

Information:

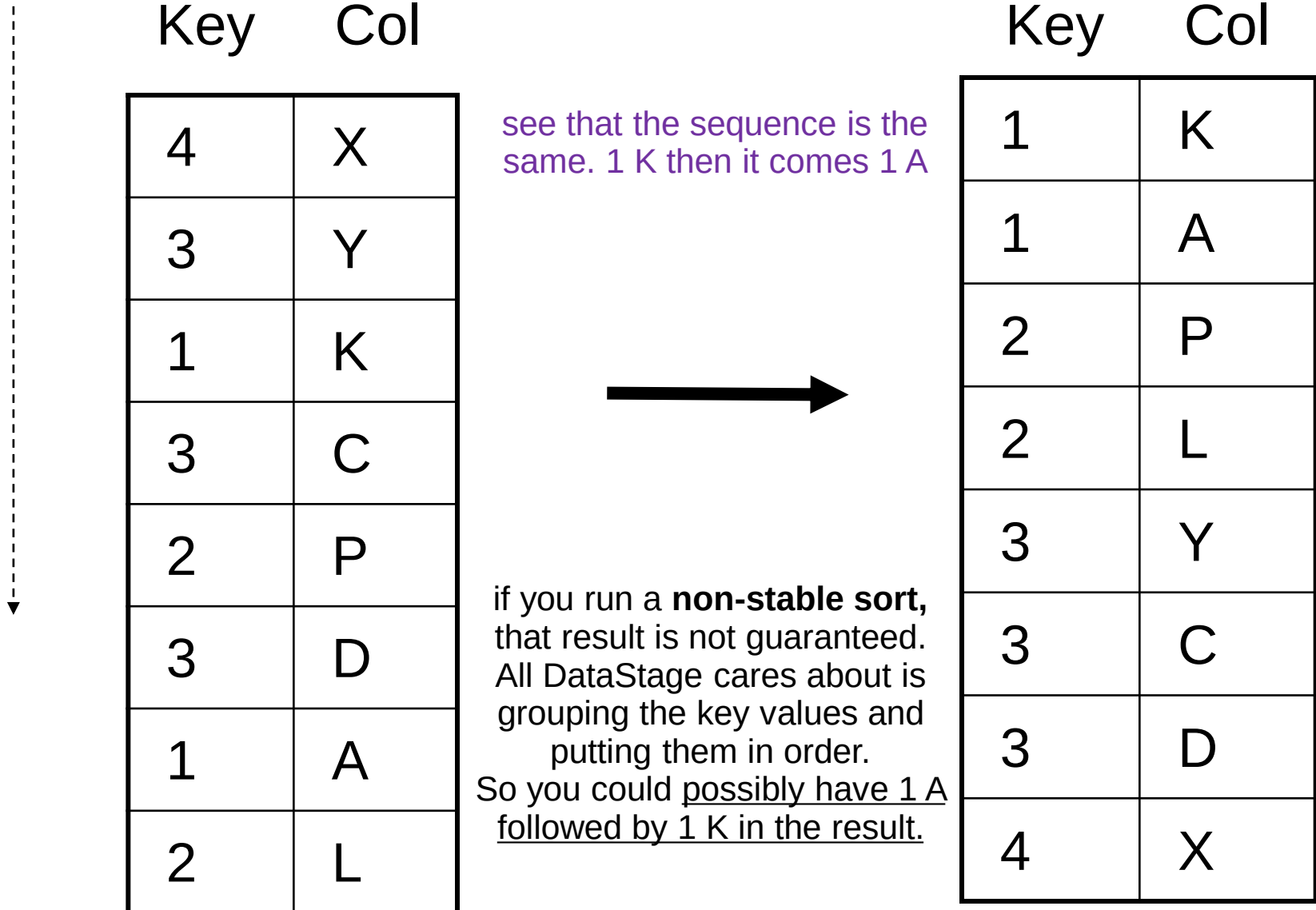
Available properties to add:

- Key

Sort Options

- Sort Utility
 - DataStage – the default
 - Unix: **Don't use. Slower than DataStage sort utility**
- Stable
- Allow Duplicates
- Restrict Memory Usage
 - Sorting takes advantage of the available memory for increased performance
 - Uses disk if necessary
 - Increasing amount of memory can improve performance
- Create Key Change Column
 - Add a column with a value of 1 / 0
 - 1 indicates that the key value has changed
 - 0 means that the key value hasn't changed
 - Useful for processing groups of rows in a Transformer

Stable Sort



Create (Cluster) Key Change Column

Key Col

4	X
3	Y
1	K
3	C
2	P
3	D
1	A
2	L

start by the key 2, the first row contains key change = 1, bcos this is the first time this value is seen, for the next key = 2, we have the value 0.

This is because this key is compared to the previous one and the algorithm identifies the value as the same.

Then it goes to the next row and compare the key values. 3 is different than 2, since the key changes, the value in the column will be one.

Key Col K_C

1	K	1
1	A	0
2	P	1
2	L	0
3	Y	1
3	C	0
3	D	0
4	X	1

Remove Duplicates Stage

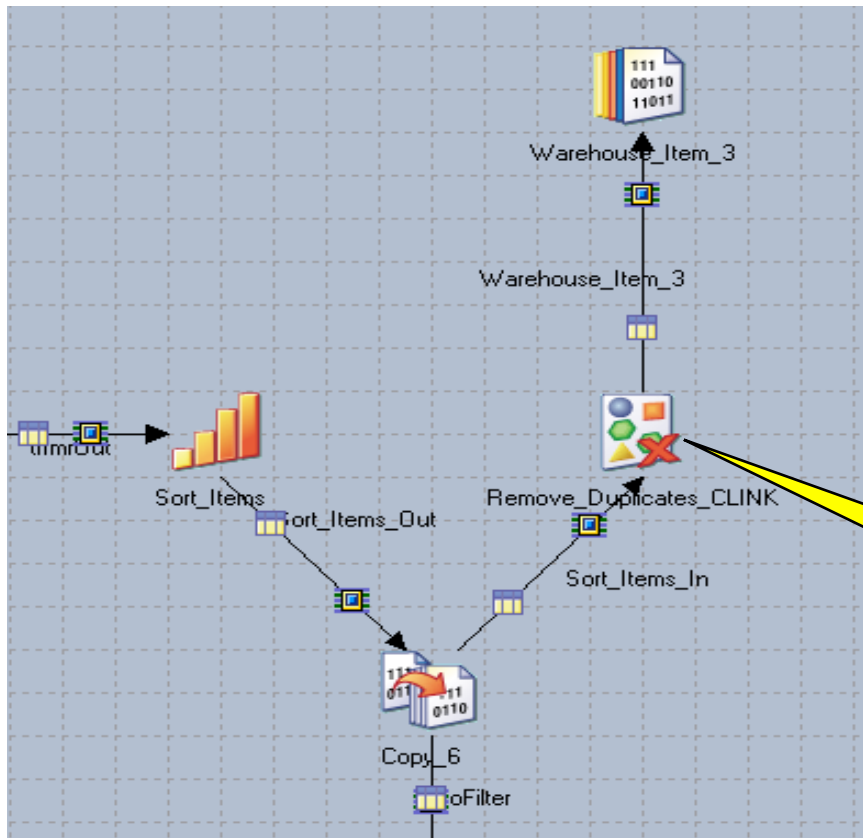
- It must have a single input link and a single output link.
- It takes a single sorted data as input, removes the duplicates and writes the data to the output.

Methods of Removing Duplicates

- Sort stage
 - Use Allow Duplicates or Unique option
 - No choice on which duplicate to keep
 - Stable sort always retains the first row in the group
 - Non-stable sort is indeterminate

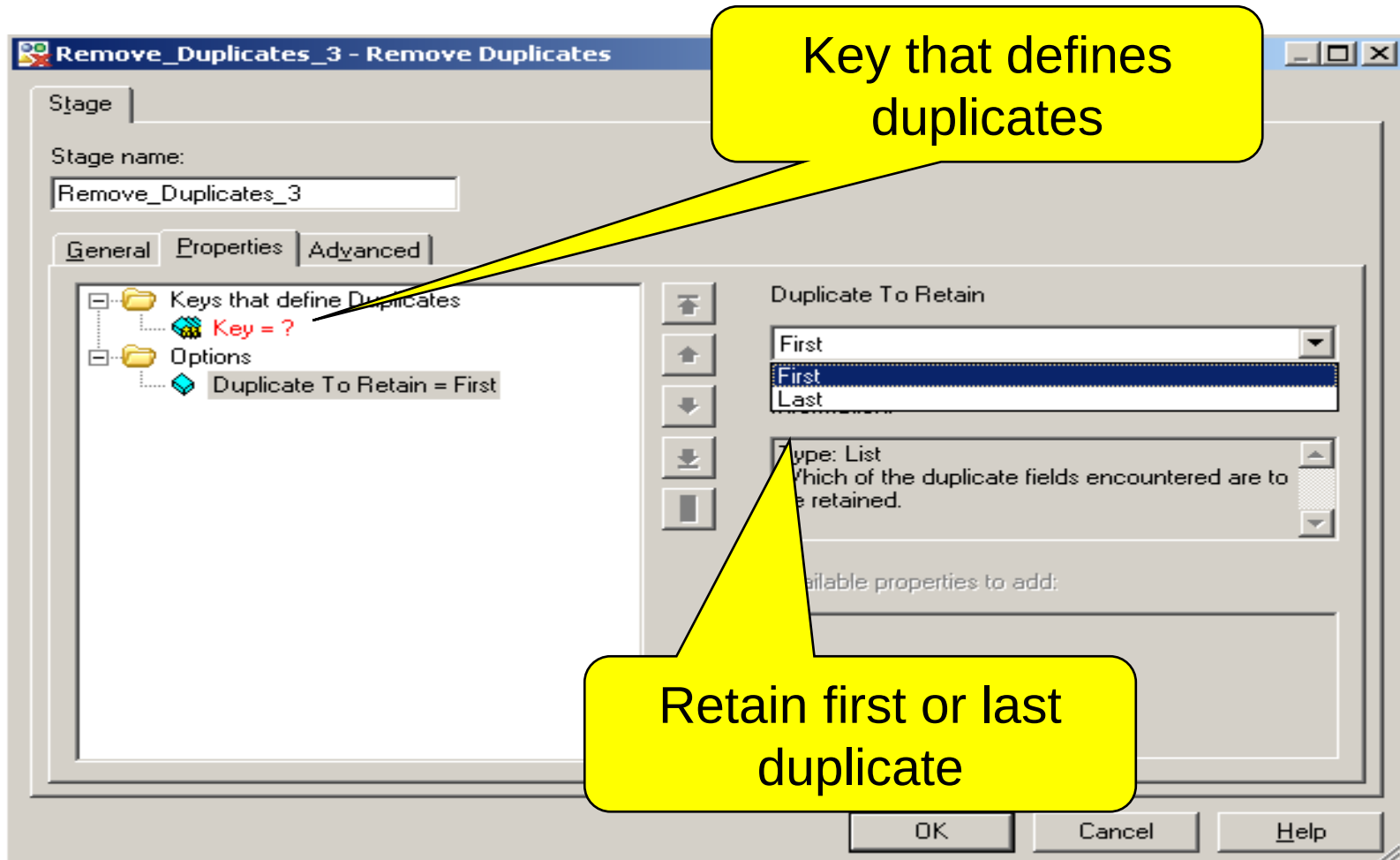
- Remove Duplicates stage
 - The prerequisite for this is sorted data as it compares the adjacent rows to identify duplicates
 - Has an option to retain first or last record

Remove Duplicates Stage Job



Remove Duplicates
stage

Remove Duplicates Stage Properties



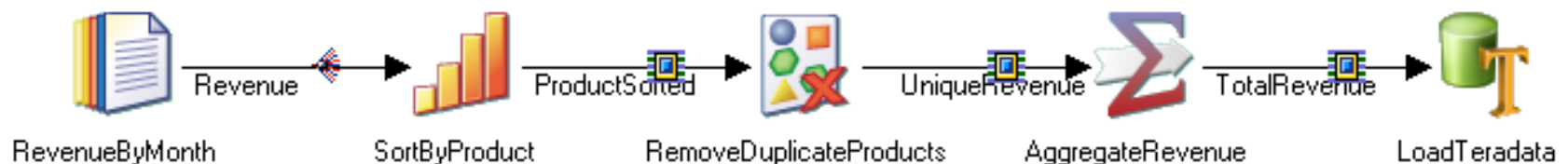
Remove Duplicates Stage Example

- Duplicate records would lead to incorrect aggregation results

Product	January Revenue
SHOVEL	3800
CAMP COT	700
ROPE 100	2600
CAMP COT	700

Output File

Sum dup Revenue	7800
Sum undup Revenue	7100



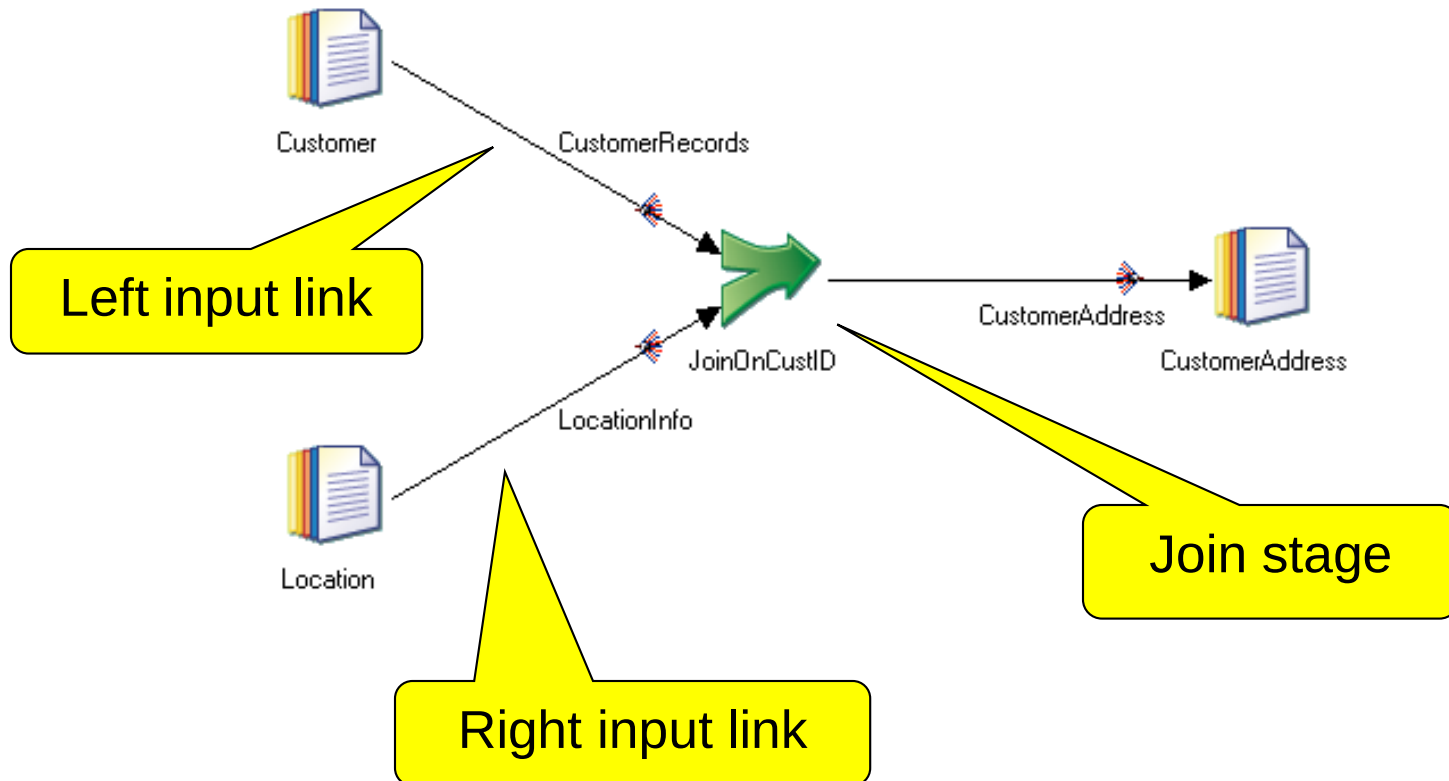
Join Stage

- The join stage performs join operations on two or more input data sets to the stage based on the key column(s) and then outputs the resulting data set.
- Unmatched rows returned in outer joins have either NULL or default values in the columns of the other links.

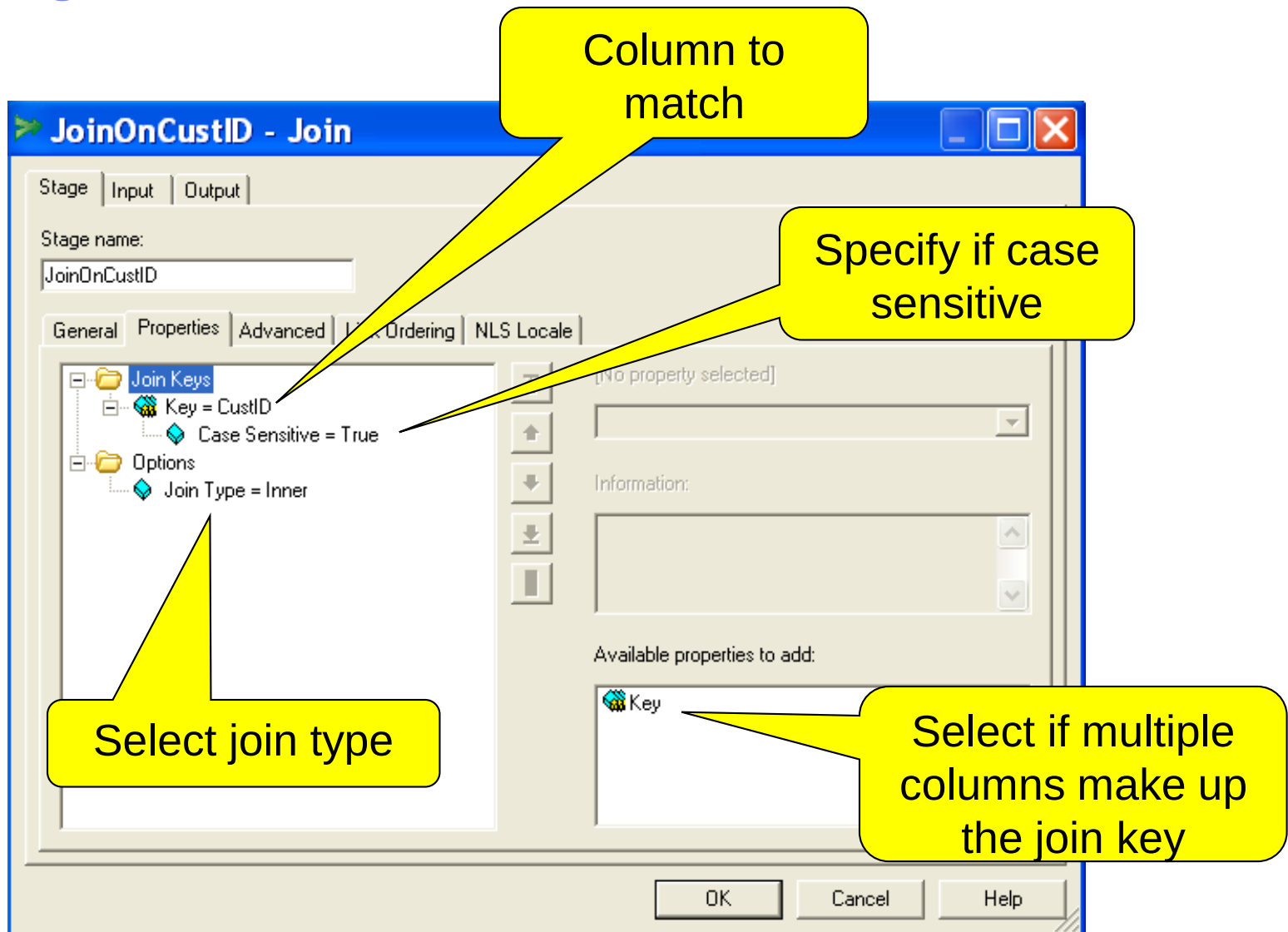
Properties of the Join Stage

- Four types of joins
 - Inner
 - Left outer
 - Right outer
 - Full outer
- Input links must be sorted
 - Left link and a right link
 - Supports additional “intermediate” links
- Light-weight
 - Little memory required, because of the sort requirement
- Join key column or columns
 - Names for each input link must match

Job With Join Stage



Join Stage Editor



Example of a Join

▪ Left link (primary input)

CustID	CustName
1543	Alpine Store
1212	Rocky Mountain
1009	Chen Bradley

Right link (secondary input)

CustID	CustType	Location
1454	Org	Bangalore, India
1212	Org	Boulder, CO, USA
1009	Ind	Bangkok, Thailand

Inner Join

- Transfers rows from both data sets whose key columns have matching values

Cust ID	Cust Name	Cust Type	Location
1212	Rocky Mountain	Org	Boulder, CO, USA
1009	Chen Bradley	Ind	Bangkok, Thailand

Left Outer Join

- Transfers all values from the left link and transfers values from the right link only where key columns match.

CustID	CustName	CustType	Location
1543	Alpine Store		
1212	Rocky Mountain	Org	Boulder, CO, USA
1009	Chen Bradley	Ind	Bangkok, Thailand

Right Outer Join

- Transfers all values from the right link and transfers values from the left link only where key columns match.

CustID	CustName	CustType	Location
1454		Org	Bangalore, India
1212	Rocky Mountain	Org	Boulder, CO, USA
1009	Chen Bradley	Ind	Bangkok, Thailand

Full Outer Join

- Transfers all rows from both data sets to the output link. The key columns may contain equal or unequal values.
- Creates new columns, with new column names!**
- Does not support more than 2 input links.**

leftRec_CustID	rightRec_CustID	CustName	CustType	Location
	1454		Org	Bangalore, India
1212	1212	Rocky Mountain	Org	Boulder, CO, USA
1009	1009	Chen Bradley	Ind	Bangkok, Thailand
1543		Alpine Store		

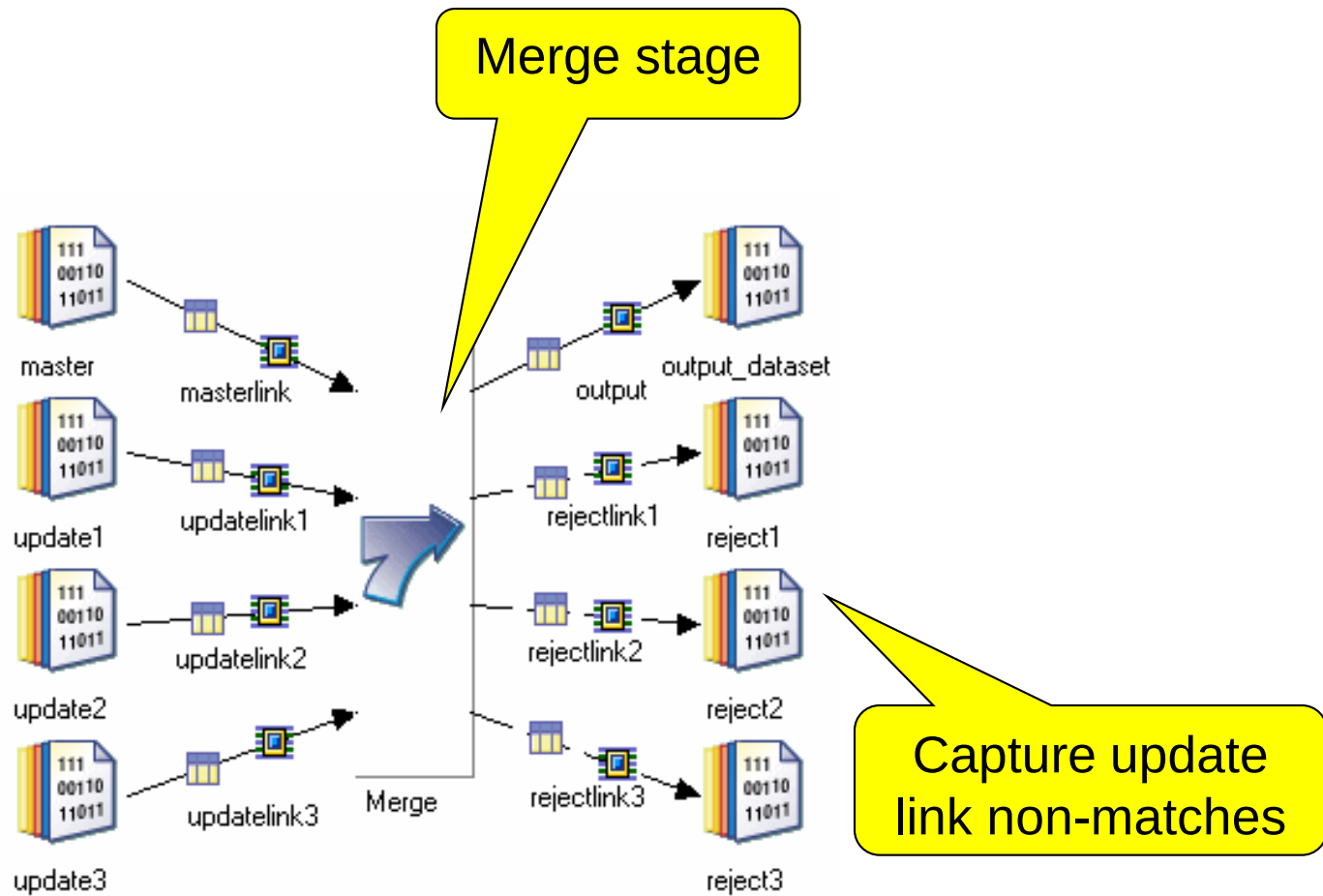
Merge Stage

- The MERGE stage copies records from all its input streams to its primary output stream.
- MERGE is intended to combine a stream record and all update records into a single output record.
- All input data (stream and updates) **must be sorted** and the **stream input cannot have duplicates**.

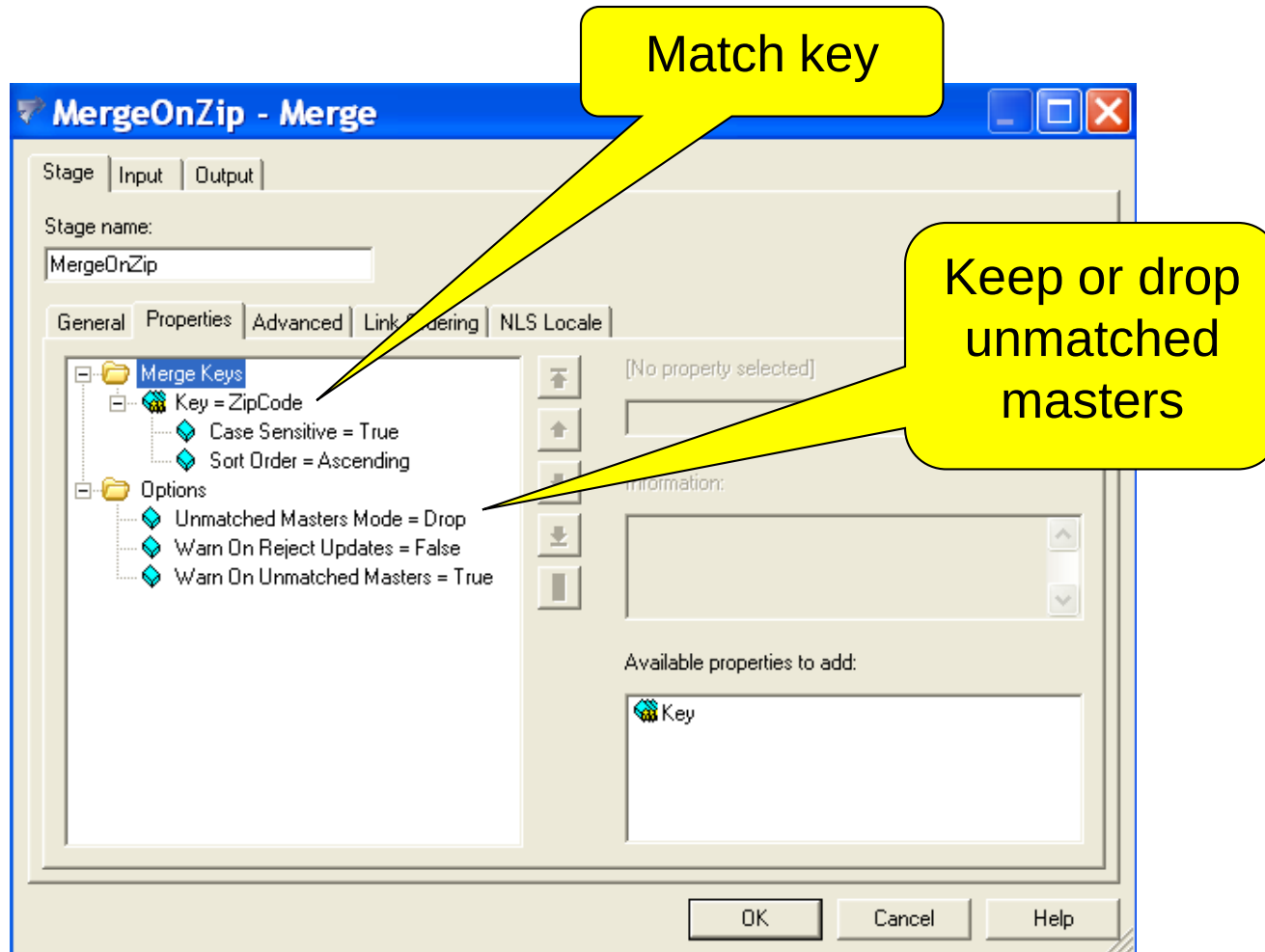
Properties of the Merge Stage

- Data inputs must be sorted and Key Partitioned, so that rows with the same key column values are located in the same partition.
- Light-weight
 - Little memory required, because of the sorting and partitioning requirement
- Unmatched master rows can be kept or dropped
- Unmatched records from secondary (update) link can be captured in a reject link

Merge Stage Job



Merge Stage Properties



Merge Stage Example

Primary Link

Emp name	Emp number	Base location	Zip Code (Key)
Sulekha	123476	Pune	411006
Sudhir	975433	Pune	411028
Albert	7765	Bangalore	560002
Vinod	447799	Pune	411057

Secondary Link

Zip Code (Key)	Addr1	Addr2
411006	IBM PUNE, Next to DON BOSCO High School	off airport road Yerwada
411028	IBM Pune, Quadra Office	Near Magarpatta City, Pune
411057	IBM Pune, ETZ Office	Marunji Pune, Maharashtra

Output of Merge with *Drop unmatched* option

Employee Name	Employee Number	Base location	Address1	Address2	Zip Code
Sulekha	123476	Pune	IBM PUNE, Next to DON BOSCO High School	off airport road Yerwada	411006
Sudhir	975433	Pune	IBM Pune, Quadra Office	Near Magarpatta City, Pune	411028
Vinod	447799	Pune	IBM Pune, ETZ Office	Marunji Pune, Maharashtra	411057

Comparison of Join, Lookup and Merge Stage

	Joins	Lookup	Merge
Model	RDBMS-style relational	Source - in RAM LU Table	Master -Update(s)
Memory usage	light	heavy	light
# and names of Inputs	2 or more: left, right	1 Source, N LU Tables	1 Master, N Update(s)
Mandatory Input Sort	all inputs	no	all inputs
Duplicates in primary input	OK	OK	Warning!
Duplicates in secondary input(s)	OK	Warning!	OK only when $N = 1$
Options on unmatched primary	Keep (left outer), Drop (Inner)	[fail] continue drop reject	[keep] drop
Options on unmatched secondary	Keep (right outer), Drop (Inner)	<i>NONE</i>	capture in reject set(s)
On match, secondary entries are	captured	captured	consumed
# Outputs	1	1 out, (1 reject)	1 out, (N rejects)
Captured in reject set(s)	<i>Nothing (N/A)</i>	unmatched primary entries	unmatched secondary entries

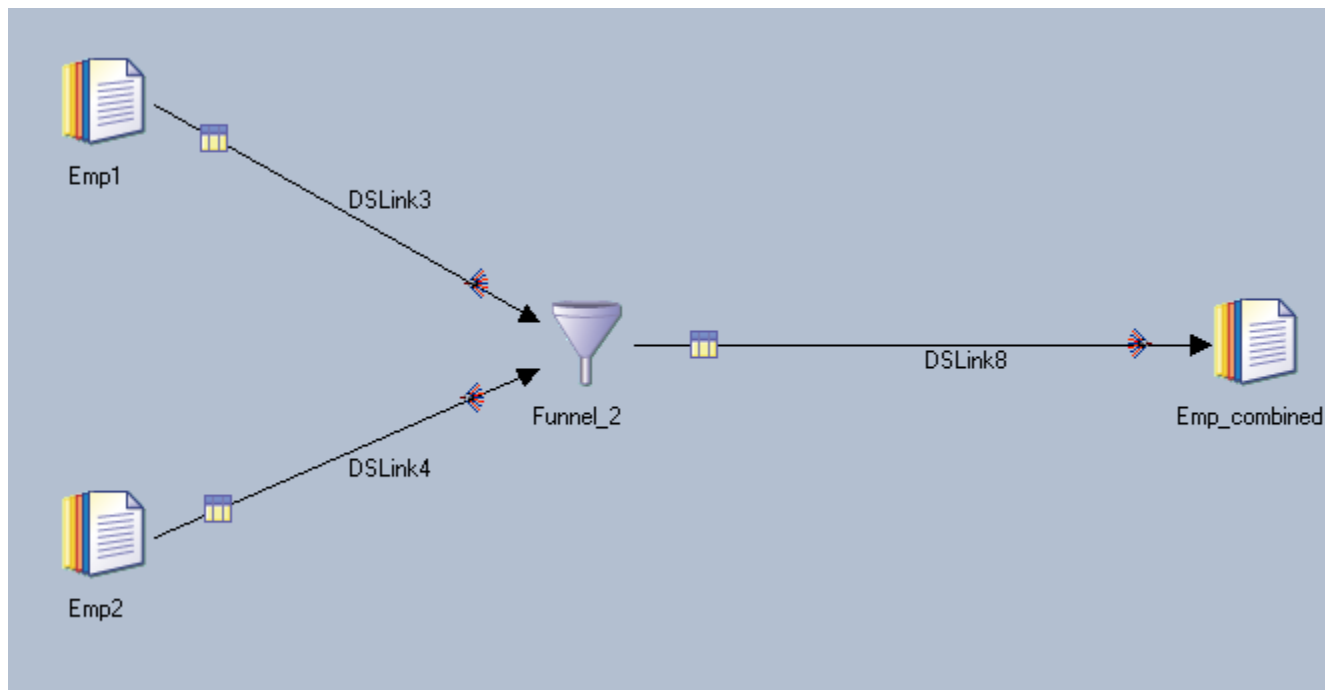
Funnel Stage

- The Funnel stage copies multiple input data stream to a single output data stream. This operation is useful for combining separate data streams into a single large data stream.
- The stage can have any number of input links and a single output link.

Properties of the Funnel Stage

- Combines data from multiple input link to a single output link.
- All sources should have identical metadata
- The three modes in which it can operate
 - Continuous
 - Records are combined in no particular order
 - First available record
 - Sequence
 - Outputs all records from the first input link
 - then all from the second input link, and so on
 - Sort Funnel
 - Produces sorted output within each partition
 - 'Passive Sort': All input data sets must have previously been sorted by the same key column(s)
 - The sort funnel uses this key to produce a sorted output

Example of the Funnel Stage



Example – Funnel Stage

Emp1

EmpID	EmpName	Location
1011	Pradeep	Bangalore
2245	Priya	Pune
2813	Lakshmi	Chennai

Emp2

EmpID	EmpName	Location
4312	Abhishek	Bangalore
1121	Priti	Pune
1342	Vijaya	Chennai

Example - Funnel Stage: Output

Emp_Combined using 'Sequence' mode

EmpID	EmpName	Location
1011	Pradeep	Bangalore
2245	Priya	Pune
2813	Lakshmi	Chennai
4312	Abhishek	Bangalore
1121	Priti	Pune
1342	Vijaya	Chennai